

CS3601 操作系统 ChCore Lab 3

王凯灵 521030910356

2023.11.22

练习 1

在 `kernel/object/cap_group.c` 中完善 `sys_create_cap_group`、`create_root_cap_group` 函数。在完成填写之后，你可以通过 Cap create pretest 测试点。

不涉及编写函数，根据提示调用所需函数即可。

练习 2

在 `kernel/object/thread.c` 中完成 `create_root_thread` 函数，将用户程序 ELF 加载到刚刚创建的进程地址空间中。

第一部分，仿照获取 TODO 上方获取 flag 的代码即可。除了似乎 `PHDR_MEMSZ_OF` 在头文件里似乎是打错了。后续的是拷贝文件、设置 flag。flag 错误并不总是影响运行，但我不是很会设置。参照 tutor 进行了修改。

练习 3

在 `kernel/arch/aarch64/sched/context.c` 中完成 `init_thread_ctx` 函数，完成线程上下文的初始化。

将要求的三个值设置为正确的值即可。

思考题 4

思考内核从完成必要的初始化到第一次切换到用户态程序的过程是怎么样的？尝试描述一下调用关系。

MMU 开启后，内核完成初始化，包括异常向量表配置等。内核从初始化完成开始（main 函数中进行 kernel test 处认为是初始化完成），首先创建第一个进程，调用 `creat_root_thread()`。创建线程时调用 `create_root_cap_group`，创建所谓的 Capability，准备资源和加载用户程序。创建完后使用调度器 `sched()` 调用线程，此后如 readme 所介绍一致。

练习 5

按照前文所述的表格填写 `kernel/arch/aarch64/irq/irq_entry.S` 中的异常向量表，并且增加对应的函数跳转操作。

填写了异常向量表，实现了相关保存恢复寄存器和栈指针切换。

练习 6

填写 `kernel/arch/aarch64/irq/irq_entry.S` 中的 `exception_enter` 与 `exception_exit`，实现上下文保存的功能，以及 `switch_to_cpu_stack` 内核栈切换函数。

不小心在上一问写掉了。同上。

思考 7

尝试描述 `printf` 如何调用到 `chcore_stdout_write` 函数。

在查找 `vprintf` 等函数时,vscode 很不幸的指向了 `/usr/include/x86_64-linux-gnu/bits/stdio2.h`。这使得递归查看十分困难。已知 `printf` 调用 `vprintf`,根据 `int vprintf(FILE *__restrict, const char *__restrict, __isoc_va_list)` 函数的定义,其中 `FILE` 为 `stdio`。`vprintf` 的实现细节难以阅读,但是显然调用在 `stdio` 中进入了 `__stdout_write`。这个函数中,调用 `syscall`。此时由于无法跳转,已经黔驴技穷,故反向出发,发现在 `stdout_ops` 中 `fd_ops` `stdout_ops` 定义 `.write = chcore_stdout_write`,再通过搜索发现在 `user/chcore-libc/libchcore/porting/overrides/src/chcore-port/syscall_dispatcher.c` 中有 `fd_dic[fd1]->fd_op = &stdout_ops;`。终于和前面接上了。内核 debug 实在是太变态了!

练习 8

在这个练习中,我们的目标是添加一行代码以完成对内核中的 `sys_putstr` 函数的系统调用。这将通过 `chcore_syscallx` 函数来实现。具体来说,您需要使用正确的系统调用编号和参数来调用 `chcore_syscallx`,以便执行 `sys_putstr`。

基于直接搜索 TODO 的方式找到这个位置。如题述,在 `put` 中添加 `chcore_syscall2(CHCORE_SYS_putstr, (long) buffer, size)` 即可。

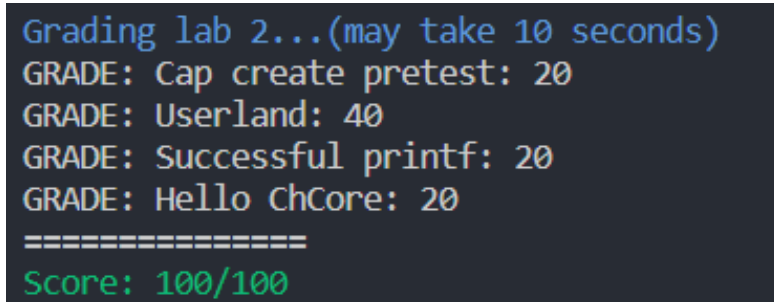
练习 9

试图编写一个简单的用户程序,其功能至少包括打印以下字符串:“Hello ChCore!”。该程序应使用 `chcore-libc` 的编译器进行编译。编译后的输出文件应命名为 `hello_chcore.bin`,并将其放入 `ramdisk` 中以内核运行。当内核启动时,它将自动运行名为 `hello_chcore.bin` 的可执行文件。

由于本次代码理解量十分庞大,仅编写最简程序。程序在提交代码的根目录中。

编译命令:`build/chcore-libc/bin/musl-gcc -o hello_chcore.bin hello_chcore.c`

至此, `make grade 100/100`



```
Grading lab 2...(may take 10 seconds)
GRADE: Cap create pretest: 20
GRADE: Userland: 40
GRADE: Successful printf: 20
GRADE: Hello ChCore: 20
=====
Score: 100/100
```

图 1

遗憾的是,似乎这里写成了 lab 2。